

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск набора подстрок в строке.

Студентка гр. 4345

Жилина А. Е.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Задание

Разработайте программу, решающую задачу точного поиска набора образцов.

Вход:

Первая строка содержит текст (T , $1 \leq |T| \leq 100000$).

Вторая - число n ($1 \leq n \leq 3000$), каждая следующая из n строк содержит шаблон из набора $P = \{p_1, \dots, p_n\}$ $1 \leq |p_i| \leq 75$

Все строки содержат символы из алфавита $\{A, C, G, T, N\}$

Выход:

Все вхождения образцов из P в T .

Каждое вхождение образца в текст представить в виде двух чисел - $i p$

Где i - позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером p (нумерация образцов начинается с 1).

Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номера шаблона.

Test data				
# Test	Input data		Output data	
1	NTAG 3 TAGT TAG T		2 2 2 3	

Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с *джокером*.

В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблоны образцу P необходимо найти все вхождения P в текст T .

Например, образец $ab??c?$ с джокером $?$ встречается дважды в тексте $xabvccbababcax$.

Символ джокер не входит в алфавит, символы которого используются в T . Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида $???$ недопустимы.

Все строки содержат символы из алфавита $\{A, C, G, T, N\}$

Вход:

Текст (T , $1 \leq |T| \leq 100000$)

Шаблон (P , $1 \leq |P| \leq 40$)

Символ джокера

Выход:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер).

Номера должны выводиться в порядке возрастания.

Test data			^
# Test	Input data	Output data	
1	ACTANCA A\$\$\$ \$	1	

Вариант 3:

Вычислить длину самой длинной цепочки из суффиксных ссылок и самой длинной цепочки из конечных ссылок в автомате.

Описание алгоритма

Для решения задачи точного поиска множества образцов и поиска одиночного шаблона с джокером реализован алгоритм на основе конечного автомата Ахо-Корасик. Основная стратегия заключается в предварительной обработке искомых подстрок с построением префиксного дерева, что позволяет сканировать текст за один проход без возвратов назад.

Реализация разделена на несколько ключевых этапов:

1. Построение автомата и расчет ссылок

Основу структуры данных составляет префиксное дерево (бор), которое формируется путем посимвольной вставки всех заданных шаблонов. После построения базовой структуры применяется обход в ширину (BFS) для создания дополнительных связей:

- Суффиксные ссылки обеспечивают быстрый переход к наибольшему совпавшему префиксу другого шаблона при несовпадении текущего символа в тексте, исключая необходимость начинать поиск заново.
- Терминальные (конечные) ссылки оптимизируют поиск вложенных образцов, указывая непосредственно на ближайший узел, который является концом какого-либо из искомых шаблонов.

В процессе обхода вершин и установки этих связей параллельно рассчитывается максимальная длина цепочек суффиксных и терминальных ссылок для выполнения требований индивидуального задания.

2. Точный множественный поиск (Задача 1)

При поиске множества образцов осуществляется последовательное чтение входного текста. На каждом шаге происходит переход по узлам автомата в соответствии с прочитанным символом. Если текущий узел является концом шаблона, фиксируется вхождение. Дополнительно происходит проход по цепочке терминальных ссылок текущего узла для извлечения всех более коротких вложенных шаблонов, заканчивающихся в этой же позиции. Итоговые результаты переводятся в 1-индексацию, сортируются по возрастанию позиции и номеру шаблона, после чего выводятся.

3. Поиск шаблона с джокером (Задача 2)

Для поиска образца со специальным символом (джокером), заменяющим ровно один любой допустимый символ алфавита, применяется метод разбиения. Исходный шаблон разделяется по символу джокера на набор значащих подстрок. Эти подстроки добавляются в автомат, при этом для каждой фиксируется её относительное смещение от начала оригинального шаблона.

Во время сканирования текста осуществляется поиск вхождений этих отдельных фрагментов. При обнаружении подстроки вычисляется потенциальная стартовая позиция всего шаблона в тексте путем вычитания относительного смещения подстроки из позиции её нахождения. Проверка целостности совпадения реализована через массив счетчиков: если для какой-либо стартовой позиции количество найденных фрагментов становится равным общему числу выделенных подстрок, это гарантирует, что все части шаблона встретились на правильном расстоянии друг от друга. Такие позиции признаются успешными и формируют итоговый ответ.

Оценка сложности алгоритма

Пусть L — суммарная длина всех обрабатываемых шаблонов (в случае поиска с джокером — длина самого шаблона без учета пропущенных символов), $|T|$ — длина входного текста, Z — общее количество найденных вхождений, а Z_f — количество найденных вхождений отдельных фрагментов при поиске с джокером.

Сложность по времени:

- **Начальный этап (Построение автомата):** Вставка всех шаблонов или выделенных значащих подстрок в префиксное дерево требует посимвольного прохода и занимает $O(L)$ времени. Расчет суффиксных и терминальных ссылок осуществляется с помощью обхода дерева в ширину (BFS). Поскольку алфавит имеет фиксированный малый размер (5 символов: A, C, G, T, N), проверка переходов дочерних узлов выполняется за константное время. Таким образом,

обработка каждого узла занимает $O(1)$, а общая асимптотика предобработки составляет $O(L)$.

- **Основной цикл (Точный множественный поиск):** Проход по тексту выполняется за один линейный скан, требуя $O(|T|)$ базовых операций. При обнаружении вхождений алгоритм извлекает шаблоны, спускаясь по цепочкам терминальных ссылок. С учетом затрат на извлечение, непосредственный поиск занимает $O(|T| + Z)$ времени. Последующая сортировка собранного массива результатов требует $O(Z * \log(Z))$ операций. Итоговое время выполнения для этой задачи в худшем случае: $O(|T| + Z * \log(Z))$.
- **Основной цикл (Поиск шаблона с джокером):** Проход автомата по тексту занимает $O(|T| + Zf)$ времени. Для каждой найденной подстроки за $O(1)$ высчитывается стартовый индекс и инкрементируется значение в массиве счетчиков `matched_pieces_count`. После завершения сканирования выполняется проход по массиву счетчиков (длиной $|T|$) для отбора позиций, где количество совпадений равно количеству подстрок, что требует $O(|T|)$ операций. Общее время этапа поиска с джокером строго укладывается в $O(|T| + Zf)$.

Сложность по памяти:

Основной постоянный объем памяти выделяется под структуру конечного автомата. Максимальное количество создаваемых узлов в дереве не превышает суммарной длины искомых строк L . Каждый узел хранит массив переходов фиксированного размера (5 ссылок) и вспомогательные переменные, что дает $O(1)$ памяти на узел. Очередь `queue`, применяемая для построения ссылок, в пиковый момент хранит узлы одного уровня дерева и требует не более $O(L)$ памяти. Следовательно, автомат потребляет $O(L)$ памяти.

Специфическое потребление памяти разделяется в зависимости от решаемой задачи:

- В задаче множественного поиска дополнительная память расходуется на список `results`, где накапливаются все найденные вхождения до момента их сортировки и вывода. Это требует $O(Z)$ дополнительной памяти. Пиковое потребление оценивается как $O(L + Z)$.

- В задаче поиска с джокером результаты поиска отдельных подстрок не аккумулируются в список. Вместо этого используется заранее инициализированный целочисленный массив счетчиков `matched_pieces_count`, размер которого равен длине текста плюс константа. Данный массив требует $O(|T|)$ памяти. Таким образом, пиковое потребление памяти алгоритмом составляет $O(L + |T|)$.

Структуры данных и ключевые переменные:

- `CHAR_TO_INDEX (dict)` — словарь для преобразования допустимых символов алфавита (A, C, G, T, N) в числовые индексы (от 0 до 4).
- `INDEX_TO_CHAR (dict)` — обратный словарь для преобразования числовых индексов обратно в символы, преимущественно используется для формирования понятных записей в лог-файле.
- `ALPHABET_LENGTH (int)` — константа, хранящая размер используемого алфавита (равна 5), применяемая для инициализации массивов переходов.
- `TrieNode (class)` — базовая структура данных, описывающая узел префиксного дерева (бора). Содержит следующие ключевые атрибуты:
 - `transitions (list)` — массив фиксированного размера (`ALPHABET_LENGTH`), хранящий ссылки на дочерние узлы для каждого возможного символа.
 - `suffix_link / terminal_link` — ссылки на узлы, обеспечивающие быстрые переходы при несовпадении символов и при поиске вложенных шаблонов.
 - `patterns (list)` — список идентификаторов (или кортежей с номером и длиной) шаблонов, заканчивающихся в текущем узле.
 - `suffix_chain / terminal_chain (int)` — целочисленные счетчики, накапливающие длину текущей цепочки суффиксных и терминальных ссылок от корня.
- `AhoCorasick (class)` — структура, инкапсулирующая корень дерева (`root`), списки узлов и глобальные переменные для хранения максимальных длин цепочек ссылок (`max_suffix_chain`, `max_terminal_chain`).

- `matched_pieces_count` (list) — массив, используемый в задаче поиска с джокером. Индекс массива соответствует потенциальной стартовой позиции всего шаблона в тексте, а значение — количеству успешно совпавших фрагментов без масок.
- `total_pattern_pieces` (int) — целочисленная переменная, фиксирующая итоговое количество значащих фрагментов, на которые был разбит исходный шаблон с джокером.

Функции алгоритма:

- `def write_log(msg)`: Глобальная утилитарная функция для ведения журнала работы. Открывает файл (`log_1.txt` или `log_2.txt`) в режиме дозаписи и сохраняет текстовую информацию о текущем состоянии структур и ходе обхода графа.
 - Аргументы:
 - `msg` — строковое сообщение, которое необходимо записать в лог-файл.
- `def add_pattern(self, pattern, pattern_id)`: Метод класса `AhoCorasick` для вставки нового шаблона (или его фрагмента) в бор. Выполняет посимвольный спуск от корня, при отсутствии необходимого перехода создает новый объект `TrieNode`. В конечном узле сохраняет переданный идентификатор.
 - Аргументы:
 - `pattern` — строка, представляющая добавляемый шаблон или подстроку.
 - `pattern_id` — идентификатор шаблона (в реализации передается кортеж, содержащий порядковый номер/стартовую позицию и длину строки).
- `def build_automaton(self)`: Метод инициализации суффиксных и терминальных ссылок, превращающий бор в конечный автомат. Работает на основе алгоритма поиска в ширину (BFS) с использованием очереди (deque). Для каждого узла вычисляет ссылки, опираясь на ссылки родительского узла, а также оптимизирует отсутствующие прямые переходы, перенаправляя их по суффиксным ссылкам. Не принимает аргументов (кроме `self`).
- `def _log_automaton_state(self)`: Вспомогательный метод для отладки. Проходит по списку всех сохраненных узлов (`self.all_nodes`) и выгружает в лог-файл сводную таблицу с информацией об их суффиксных и терминальных ссылках, переходах и хранящихся шаблонах. Не принимает аргументов.

- `def search(self, text):` Основной метод обхода текста (реализован как генератор). Посимвольно сканирует входную строку, совершая переходы по узлам автомата. При обнаружении шаблонов в текущем узле или по цепочке терминальных ссылок возвращает (через `yield`) позицию окончания и идентификатор найденного образца.
 - Аргументы:
 - `text` — входная текстовая строка, в которой осуществляется поиск.

Тестирование

Таблица 1 – Результаты тестирования (точный поиск)

№ п/п	Входные данные	Выходные данные	Комментарии
1.	ACGTNACGTN 3 ACG TN ACGTN	1 1 1 3 4 2 6 1 6 3 9 2 Длина самой длинной цепочки из суффиксных ссылок:2 Длина самой длинной цепочки из терминальных ссылок:1	Верный результат. Найдены все вхождения, включая перекрытия.
2.	AAAAA 3 A AA AAA	1 1 1 2 1 3 2 1 2 2 2 3 3 1 3 2 3 3 4 1 4 2	Верный результат. Проверена глубокая вложенность и работа терминальных ссылок.

		5 1 Длина самой длинной цепочки из суффиксных ссылок:3 Длина самой длинной цепочки из терминальных ссылок:2	
3.	GTCANNTG 2 TCA GTCAN	1 2 2 1 Длина самой длинной цепочки из суффиксных ссылок:2 Длина самой длинной цепочки из терминальных ссылок:1	Верный результат. Корректная отработка паттерна, содержащего символ 'N'.
4.	NNTTAGGCAC 4 NN TT GG AC	1 1 3 2 6 3 9 4 Длина самой длинной цепочки из суффиксных ссылок:2 Длина самой длинной цепочки из терминальных ссылок:0	Верный результат. Проверка работы по всем символам алфавита.

Таблица 2 – Результаты тестирования (поиск с джокером)

№ п/п	Входные данные	Выходные данные	Комментарии
1	ACGTACG A?G ?	1 5 Длина самой длинной цепочки из суффиксных ссылок:1 Длина самой длинной цепочки из терминальных ссылок:0	Корректная обработка одиночного джокера в центральной позиции шаблона.
2	GNTACG !N!	1 Длина самой длинной	Проверка работы алгоритма при

	!	цепочки из суффиксных ссылок:1 Длина самой длинной цепочки из терминальных ссылок:0	расположении джокеров на границах искомого образца.
3	NTAGNTAG NT#GN #	1 Длина самой длинной цепочки из суффиксных ссылок:2 Длина самой длинной цепочки из терминальных ссылок:0	Тест на разделение шаблона на фрагменты при наличии джокера в специфической позиции.
4	ACGTACGT A..T .	1 5 Длина самой длинной цепочки из суффиксных ссылок:1 Длина самой длинной цепочки из терминальных ссылок:0	Успешная обработка последовательности из нескольких символов- джокеров (пустые фрагменты между ними игнорируются).

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: task1.py

```
from collections import deque

# Очистка лог-файла перед началом работы
with open("log_1.txt", "w", encoding="utf-8") as f:
    f.write("[ИНИЦИАЛИЗАЦИЯ] Запуск алгоритма Ахо-Корасик\n")
    f.write("=" * 70 + "\n")

def write_log(msg):
    with open("log_1.txt", "a", encoding="utf-8") as f:
        f.write(msg + "\n")

# Фиксированный алфавит и размер словаря
CHAR_TO_INDEX = {'A': 0, 'C': 1, 'G': 2, 'T': 3, 'N': 4}
INDEX_TO_CHAR = {0: 'A', 1: 'C', 2: 'G', 3: 'T', 4: 'N'}
ALPHABET_LENGTH = 5

class TrieNode:
    _id_counter = 0 # Глобальный счетчик для нумерации узлов в логах

    def __init__(self):
        self.id = TrieNode._id_counter
        TrieNode._id_counter += 1

        # Переходы к дочерним узлам для каждого символа алфавита
        self.transitions = [None] * ALPHABET_LENGTH
        self.suffix_link = None
        self.terminal_link = None

        # Идентификаторы шаблонов и счетчики длин цепочек
        self.patterns = []
        self.suffix_chain = 0
        self.terminal_chain = 0

class AhoCorasick:
    def __init__(self):
        TrieNode._id_counter = 0
        self.root = TrieNode()
        self.max_suffix_chain = 0
        self.max_terminal_chain = 0

        # Сохраняем все узлы для удобного вывода состояния автомата в лог
        self.all_nodes = [self.root]

    def add_pattern(self, pattern, pattern_id):
        curr = self.root
        write_log(f"\n[ПОСТРОЕНИЕ БОРА] Вставка шаблона: '{pattern}'. ID: {pattern_id[0]}, Длина: {pattern_id[1]}")
        prefix = ""

        # Спускаемся по дереву, создавая узлы при необходимости
        for char in pattern:
            idx = CHAR_TO_INDEX[char]
```

```

        prefix += char

        if curr.transitions[idx] is None:
            curr.transitions[idx] = TrieNode()
            self.all_nodes.append(curr.transitions[idx])
            write_log(
                f" -- Символ '{char}' (префикс '{prefix}'): перехода из
узла {curr.id} нет. Создан новый узел {curr.transitions[idx].id}. Переходим в
него.")
        else:
            write_log(
                f" -- Символ '{char}' (префикс '{prefix}'): переход из узла
{curr.id} уже существует. Переходим в узел {curr.transitions[idx].id}.")
            curr = curr.transitions[idx]

        # Добавляем шаблон в список patterns его конечного узла
        curr.patterns.append(pattern_id)
        write_log(f" -> Конец шаблона '{pattern}' сохранён в узле {curr.id}.")

def build_automaton(self):
    write_log("\n" + "=" * 70)
    write_log("[ПОСТРОЕНИЕ АВТОМАТА] Шаг 1: Инициализация первого уровня")
    write_log("=" * 70)

    queue = deque()
    self.root.suffix_link = self.root
    write_log(" -- Суффиксная ссылка корня (узел 0) замкнута на себя (узел
0).")

    # Инициализируем первый уровень, связывая его с корнем
    for i in range(ALPHABET_LENGTH):
        child = self.root.transitions[i]
        char = INDEX_TO_CHAR[i]
        if child is not None:
            child.suffix_link = self.root
            child.suffix_chain = 1
            self.max_suffix_chain = max(self.max_suffix_chain, 1)
            queue.append(child)
            write_log(
                f" -- Прямой потомок корня: узел {child.id} (по символу
'{char}'). Суф. ссылка направлена на корень (0). Узел добавлен в очередь.")
        else:
            self.root.transitions[i] = self.root
            write_log(f" -- Отсутствует переход из корня по '{char}'.
Замыкаем переход на корень (0).")

    write_log("\n" + "=" * 70)
    write_log("[ПОСТРОЕНИЕ АВТОМАТА] Шаг 2: Обход в ширину (BFS) для
остальных уровней")
    write_log("=" * 70)

    # Строим ссылки через BFS для корректной обработки суффиксов
    while queue:
        curr = queue.popleft()
        write_log(f"\n [ОБРАБОТКА УЗЛА {curr.id}] (суф. ссылка указывает на
узел {curr.suffix_link.id})")

        # Устанавливаем терминальную ссылку для быстрого поиска вхождений
        if curr.suffix_link.patterns:
            curr.terminal_link = curr.suffix_link
            curr.terminal_chain = curr.suffix_link.terminal_chain + 1
            write_log(

```

```

        f"      -- Терминальная ссылка: суффиксная ссылка (узел
{curr.suffix_link.id}) является концом шаблона. Устанавливаем терм. ссылку
{curr.id} -> {curr.suffix_link.id}.")
    else:
        curr.terminal_link = curr.suffix_link.terminal_link
        curr.terminal_chain = curr.suffix_link.terminal_chain
        if curr.terminal_link:
            write_log(
                f"      -- Терминальная ссылка: унаследована от суф.
ссылки (узла {curr.suffix_link.id}). Направлена на узел
{curr.terminal_link.id}.")
        else:
            write_log(
                f"      -- Терминальная ссылка: отсутствует (суффиксная
ссылка не ведет к концу шаблона и не имеет своей терм. ссылки).")

        self.max_terminal_chain = max(self.max_terminal_chain,
curr.terminal_chain)

    # Настраиваем переходы и суффиксные ссылки для потомков
    for i in range(ALPHABET_LENGTH):
        child = curr.transitions[i]
        char = INDEX_TO_CHAR[i]

        if child is not None:
            # Суффиксная ссылка указывает на переход родителя
            child.suffix_link = curr.suffix_link.transitions[i]
            child.suffix_chain = child.suffix_link.suffix_chain + 1
            self.max_suffix_chain = max(self.max_suffix_chain,
child.suffix_chain)
            queue.append(child)
            write_log(
                f"      -- Найден потомок по '{char}' (узел {child.id}).
Переход из суф. ссылки родителя (узла {curr.suffix_link.id}) по '{char}' ведет в
узел {curr.suffix_link.transitions[i].id}. Устанавливаем суф. ссылку {child.id}
-> {child.suffix_link.id}. Узел добавлен в очередь.")
        else:
            # Оптимизируем переходы, перенаправляя их через суффиксную
ссылку
            curr.transitions[i] = curr.suffix_link.transitions[i]
            write_log(
                f"      -- Отсутствует прямой переход по '{char}'.
Достраиваем граф переходов автомата: перенаправляем по суф. ссылке в узел
{curr.transitions[i].id}.")

    self._log_automaton_state()

    def _log_automaton_state(self):
        write_log("\n" + "=" * 70)
        write_log("[СОСТОЯНИЕ ИТОГОВОГО АВТОМАТА]")
        write_log("=" * 70)
        for node in self.all_nodes:
            trans = ", ".join([f"'{INDEX_TO_CHAR[i]}':{t.id if t else '0'}" for
i, t in enumerate(node.transitions)])
            suflink = node.suffix_link.id if node.suffix_link else "None"
            termlink = node.terminal_link.id if node.terminal_link else "None"
            write_log(
                f"      Узел {node.id:02d} | Суф. ссылка: {suflink:>4} | Терм.
ссылка: {termlink:>4} | Шаблоны: {node.patterns}")
            write_log(f"      | Переходы: {trans}")
            write_log("=" * 70 + "\n")

    def search(self, text):

```

```

write_log(f"[ПОИСК] Старт обхода текста: '{text}'\n")
curr = self.root

# Сканируем текст, переходя по узлам автомата
for i, char in enumerate(text):
    if char not in CHAR_TO_INDEX:
        write_log(f"-- Шаг {i}: символ '{char}' вне алфавита,
игнорируем.")
        continue

    idx = CHAR_TO_INDEX[char]
    next_node = curr.transitions[idx]
    write_log(f"-- Шаг {i}: считываем '{char}'. Переход: узел
{curr.id} -> узел {next_node.id}")
    curr = next_node

    # Сообщаем о шаблонах, найденных непосредственно в этом узле
    if curr.patterns:
        for pid in curr.patterns:
            write_log(f"--> [СОВПАДЕНИЕ] Конец шаблона {pid[0]}
обнаружен прямо в текущем узле {curr.id}.")
            yield (i, pid)

# Проходим по цепочке терминальных ссылок для поиска вложенных
шаблонов
term_node = curr.terminal_link
if term_node is not None:
    write_log(
        f"--> [ПРОВЕРКА] Обнаружена терминальная ссылка на узел
{term_node.id}. Спуск для проверки вложенных подстрок...")

    while term_node is not None:
        for pid in term_node.patterns:
            write_log(
                f"--> [ВЛОЖЕННОЕ СОВПАДЕНИЕ] Шаблон {pid[0]}
извлечен из терминального узла {term_node.id}.")
            yield (i, pid)
        term_node = term_node.terminal_link

write_log("\n[ЗАВЕРШЕНИЕ] Конец текста достигнут.")

# Считываем текст и количество шаблонов
text = input()
n = int(input())

ac = AhoCorasick()

# Построчно считываем каждый шаблон
# и сохраняем кортеж (номер, длина) для вычисления стартовой позиции
for i in range(n):
    pattern = input()
    ac.add_pattern(pattern, (i + 1, len(pattern)))

# Инициализируем суффиксные и терминальные ссылки
ac.build_automaton()

results = []

# Проходим по тексту и собираем найденные вхождения
for end_pos, (p_num, p_len) in ac.search(text):

```

```

# Переход от 0-индексации конца к 1-индексации начала
start_pos = end_pos - p_len + 2
results.append((start_pos, p_num))

# Сортировка по возрастанию позиции, при равенстве - по номеру
results.sort()

# Выводим итоговый результат
for start_pos, p_num in results:
    print(f"{start_pos} {p_num}")

```

Название файла: task2.py

```

from collections import deque

# Очистка лог-файла перед началом работы
with open("log_2.txt", "w", encoding="utf-8") as f:
    f.write("[ИНИЦИАЛИЗАЦИЯ] Запуск алгоритма Ахо-Корасик\n")
    f.write("=" * 70 + "\n")

def write_log(msg):
    with open("log_2.txt", "a", encoding="utf-8") as f:
        f.write(msg + "\n")

# Фиксированный алфавит и размер словаря
CHAR_TO_INDEX = {'A': 0, 'C': 1, 'G': 2, 'T': 3, 'N': 4}
INDEX_TO_CHAR = {0: 'A', 1: 'C', 2: 'G', 3: 'T', 4: 'N'}
ALPHABET_LENGTH = 5

class TrieNode:
    _id_counter = 0 # Глобальный счетчик для нумерации узлов в логах

    def __init__(self):
        self.id = TrieNode._id_counter
        TrieNode._id_counter += 1

        # Переходы к дочерним узлам для каждого символа алфавита
        self.transitions = [None] * ALPHABET_LENGTH
        self.suffix_link = None
        self.terminal_link = None

        # Идентификаторы шаблонов и счетчики длин цепочек
        self.patterns = []
        self.suffix_chain = 0
        self.terminal_chain = 0

class AhoCorasick:
    def __init__(self):
        TrieNode._id_counter = 0
        self.root = TrieNode()
        self.max_suffix_chain = 0
        self.max_terminal_chain = 0

        # Сохраняем все узлы для удобного вывода состояния автомата в лог
        self.all_nodes = [self.root]

    def add_pattern(self, pattern, pattern_id):

```

```

curr = self.root
write_log(f"\n[ПОСТРОЕНИЕ БОРА] Вставка шаблона: '{pattern}'. ID:
{pattern_id[0]}, Длина: {pattern_id[1]}")
prefix = ""

# Спускаемся по дереву, создавая узлы при необходимости
for char in pattern:
    idx = CHAR_TO_INDEX[char]
    prefix += char

    if curr.transitions[idx] is None:
        curr.transitions[idx] = TrieNode()
        self.all_nodes.append(curr.transitions[idx])
        write_log(
            f" -- Символ '{char}' (префикс '{prefix}'): перехода из
узла {curr.id} нет. Создан новый узел {curr.transitions[idx].id}. Переходим в
него.")
    else:
        write_log(
            f" -- Символ '{char}' (префикс '{prefix}'): переход из узла
{curr.id} уже существует. Переходим в узел {curr.transitions[idx].id}.")
        curr = curr.transitions[idx]

# Добавляем шаблон в список patterns его конечного узла
curr.patterns.append(pattern_id)
write_log(f" -> Конец шаблона '{pattern}' сохранён в узле {curr.id}.")

def build_automaton(self):
    write_log("\n" + "=" * 70)
    write_log("[ПОСТРОЕНИЕ АВТОМАТА] Шаг 1: Инициализация первого уровня")
    write_log("=" * 70)

    queue = deque()
    self.root.suffix_link = self.root
    write_log(" -- Суффиксная ссылка корня (узел 0) замкнута на себя (узел
0).")

    # Инициализируем первый уровень, связывая его с корнем
    for i in range(ALPHABET_LENGTH):
        child = self.root.transitions[i]
        char = INDEX_TO_CHAR[i]
        if child is not None:
            child.suffix_link = self.root
            child.suffix_chain = 1
            self.max_suffix_chain = max(self.max_suffix_chain, 1)
            queue.append(child)
            write_log(
                f" -- Прямой потомок корня: узел {child.id} (по символу
'{char}'). Суф. ссылка направлена на корень (0). Узел добавлен в очередь.")
        else:
            self.root.transitions[i] = self.root
            write_log(f" -- Отсутствует переход из корня по '{char}'.
Замыкаем переход на корень (0).")

    write_log("\n" + "=" * 70)
    write_log("[ПОСТРОЕНИЕ АВТОМАТА] Шаг 2: Обход в ширину (BFS) для
остальных уровней")
    write_log("=" * 70)

    # Строим ссылки через BFS для корректной обработки суффиксов
    while queue:
        curr = queue.popleft()

```

```

        write_log(f"\n [ОБРАБОТКА УЗЛА {curr.id}] (суф. ссылка указывает на
узел {curr.suffix_link.id})")

        # Устанавливаем терминальную ссылку для быстрого поиска вхождений
        if curr.suffix_link.patterns:
            curr.terminal_link = curr.suffix_link
            curr.terminal_chain = curr.suffix_link.terminal_chain + 1
            write_log(
                f" -- Терминальная ссылка: суффиксная ссылка (узел
{curr.suffix_link.id}) является концом шаблона. Устанавливаем терм. ссылку
{curr.id} -> {curr.suffix_link.id}.")
        else:
            curr.terminal_link = curr.suffix_link.terminal_link
            curr.terminal_chain = curr.suffix_link.terminal_chain
            if curr.terminal_link:
                write_log(
                    f" -- Терминальная ссылка: унаследована от суф.
ссылки (узла {curr.suffix_link.id}). Направлена на узел
{curr.terminal_link.id}.")
            else:
                write_log(
                    f" -- Терминальная ссылка: отсутствует (суффиксная
ссылка не ведет к концу шаблона и не имеет своей терм. ссылки).")

        self.max_terminal_chain = max(self.max_terminal_chain,
curr.terminal_chain)

        # Настраиваем переходы и суффиксные ссылки для потомков
        for i in range(ALPHABET_LENGTH):
            child = curr.transitions[i]
            char = INDEX_TO_CHAR[i]

            if child is not None:
                # Суффиксная ссылка указывает на переход родителя
                child.suffix_link = curr.suffix_link.transitions[i]
                child.suffix_chain = child.suffix_link.suffix_chain + 1
                self.max_suffix_chain = max(self.max_suffix_chain,
child.suffix_chain)
                queue.append(child)
                write_log(
                    f" -- Найден потомок по '{char}' (узел {child.id}).
Переход из суф. ссылки родителя (узла {curr.suffix_link.id}) по '{char}' ведет в
узел {curr.suffix_link.transitions[i].id}. Устанавливаем суф. ссылку {child.id}
-> {child.suffix_link.id}. Узел добавлен в очередь.")
            else:
                # Оптимизируем переходы, перенаправляя их через суффиксную
ссылку
                curr.transitions[i] = curr.suffix_link.transitions[i]
                write_log(
                    f" -- Отсутствует прямой переход по '{char}'.
Достраиваем граф переходов автомата: перенаправляем по суф. ссылке в узел
{curr.transitions[i].id}.")

        self._log_automaton_state()

    def _log_automaton_state(self):
        write_log("\n" + "=" * 70)
        write_log("[СОСТОЯНИЕ ИТОГОВОГО АВТОМАТА]")
        write_log("=" * 70)
        for node in self.all_nodes:
            trans = ", ".join([f"'{INDEX_TO_CHAR[i]}':{t.id if t else '0'}" for
i, t in enumerate(node.transitions)])
            suflink = node.suffix_link.id if node.suffix_link else "None"

```

```

        termlink = node.terminal_link.id if node.terminal_link else "None"
        write_log(
            f" Узел {node.id:02d} | Суф. ссылка: {suflink:>4} | Терм.
ссылка: {termlink:>4} | Шаблоны: {node.patterns}")
        write_log(f" | Переходы: {trans}")
        write_log("=" * 70 + "\n")

    def search(self, text):
        write_log(f"[ПОИСК] Старт обхода текста: '{text}'\n")
        curr = self.root

        # Сканируем текст, переходя по узлам автомата
        for i, char in enumerate(text):
            if char not in CHAR_TO_INDEX:
                write_log(f" -- Шаг {i}: символ '{char}' вне алфавита,
игнорируем.")
                continue

            idx = CHAR_TO_INDEX[char]
            next_node = curr.transitions[idx]
            write_log(f" -- Шаг {i}: считываем '{char}'. Переход: узел
{curr.id} -> узел {next_node.id}")
            curr = next_node

            # Сообщаем о шаблонах, найденных непосредственно в этом узле
            if curr.patterns:
                for pid in curr.patterns:
                    write_log(f" -> [СОВПАДЕНИЕ] Конец шаблона {pid[0]}
обнаружен прямо в текущем узле {curr.id}.")
                    yield (i, pid)

            # Проходим по цепочке терминальных ссылок для поиска вложенных
шаблонов
            term_node = curr.terminal_link
            if term_node is not None:
                write_log(
                    f" -> [ПРОВЕРКА] Обнаружена терминальная ссылка на узел
{term_node.id}. Спуск для проверки вложенных подстрок...")

                while term_node is not None:
                    for pid in term_node.patterns:
                        write_log(
                            f" -> [ВЛОЖЕННОЕ СОВПАДЕНИЕ] Шаблон {pid[0]}
извлечен из терминального узла {term_node.id}.")
                        yield (i, pid)
                    term_node = term_node.terminal_link

        write_log("\n[ЗАВЕРШЕНИЕ] Конец текста достигнут.")

    # Считываем входные данные
    text = input()
    pattern = input()
    wildcard = input()

    ac = AhoCorasick()

    total_pattern_pieces = 0
    chars_processed = 0

    write_log("\n[ПРЕДОБРАБОТКА ШАБЛОНА] Разбиение по символу джокера")
    # Разбиваем шаблон на подстроки без масок, сохраняя их стартовые позиции

```

```

for piece in pattern.split(wildcard):
    if piece:
        piece_start_pos = chars_processed + 1
        ac.add_pattern(piece, (piece_start_pos, len(piece)))
        total_pattern_pieces += 1
        write_log(f" -- Выделена подстрока: '{piece}'. Позиция в шаблоне:
{piece_start_pos}, Длина: {len(piece)}")
    else:
        write_log(" -- Пропуск пустого блока (подряд идущие джокеры или джокер
с краем).")
        chars_processed += len(piece) + 1

write_log(f" -> Итого выделено значащих подстрок: {total_pattern_pieces}")

# Строим автомат для поиска подстрок
ac.build_automaton()

# Массив, где индекс – потенциальная стартовая позиция шаблона в тексте,
# а значение – количество совпавших частей без масок
matched_pieces_count = [0] * (len(text) + 2)

write_log("\n" + "=" * 70)
write_log("[ОБРАБОТКА РЕЗУЛЬТАТОВ ПОИСКА] Подсчет совпадений для потенциальных
стартовых позиций")
write_log("=" * 70)

# Ищем вхождения всех подстрок без масок
for end_pos, (piece_start_in_pattern, piece_length) in ac.search(text):
    found_piece_start_in_text = end_pos - piece_length + 2
    candidate_pattern_start = found_piece_start_in_text - piece_start_in_pattern
    + 1

    # Если потенциальное начало шаблона попадает в допустимые границы текста
    if 1 <= candidate_pattern_start < len(matched_pieces_count):
        matched_pieces_count[candidate_pattern_start] += 1
        write_log(f" -- Найдена подстрока (стартовая позиция в шаблоне
{piece_start_in_pattern}). "
f"Ожидаемое начало всего шаблона в тексте:
{candidate_pattern_start}). "
f"Счетчик для позиции {candidate_pattern_start} увеличен до
{matched_pieces_count[candidate_pattern_start]}")
    else:
        write_log(f" -- Найдена подстрока (стартовая позиция в шаблоне
{piece_start_in_pattern}), но "
f"ожидаемое начало всего шаблона ({candidate_pattern_start})
выходит за границы допустимого. Игнорируем.")

write_log("\n" + "=" * 70)
write_log(f"[ФОРМИРОВАНИЕ ОТВЕТА] Отбор позиций, где совпали все
{total_pattern_pieces} подстрок")
write_log("=" * 70)

# Выводим позиции, где количество совпавших подстрок равно общему количеству
кусков
# Ограничиваемся только теми позициями, где шаблон физически влезает в текст
valid_max_start = len(text) - len(pattern) + 2
for start_idx in range(1, valid_max_start):
    if matched_pieces_count[start_idx] == total_pattern_pieces:
        write_log(f" -> [УСПЕХ] Позиция {start_idx}: совпали все подстроки.
Добавлена в итоговый ответ.")
        print(start_idx)
    elif matched_pieces_count[start_idx] > 0:

```

```
        write_log(f"    -- Позиция {start_idx}: совпало подстрок  
{matched_pieces_count[start_idx]} из {total_pattern_pieces}. Не подходит.")
```